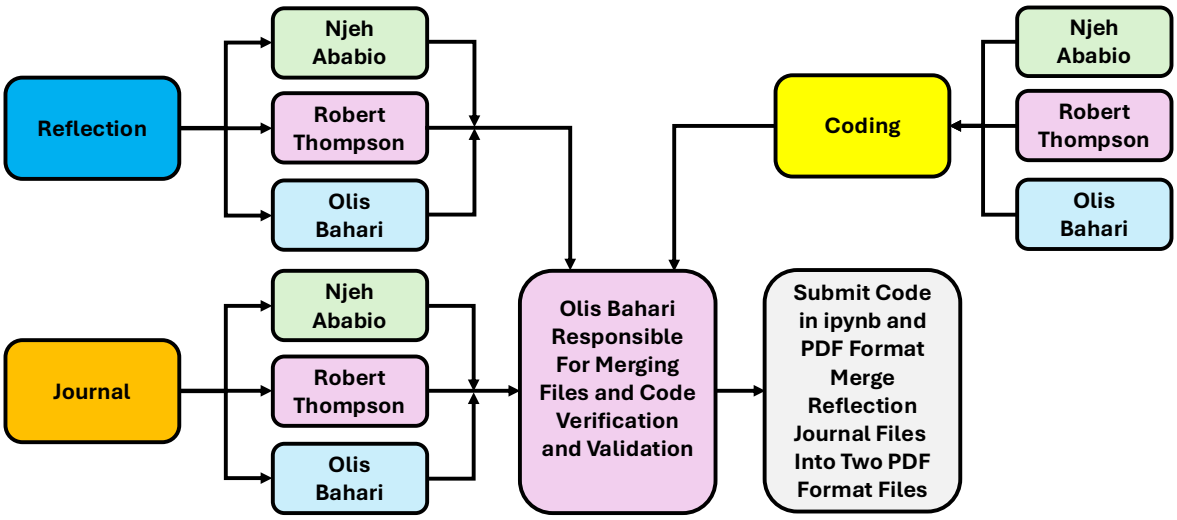
Midterm Reflection Merged Njeh, Robert, & Olis ITAI2376

Midterm Workflow Process Team Task

V4.0



Njeh Ababio: Learning Reflection on Diffusion Models Project

Through this assignment, I gained a solid understanding of how diffusion is used to produce images of deep learning concepts in a controlled context. The most important thing I learned is that the generation process is systematic. As noise is introduced during training, the model learns to predict the noise at each step, and it uses that prediction to generate the image.

When I understand this process, I realize that the forward diffusion process corrupts an image by gradually introducing random noise. During training, given a noisy image and the corresponding noise level, the model attempts to predict the added noise. The training loss is computed according to the difference between the predicted noise and the actual noise. This makes it clear how the training objective relates to the reverse process, which begins with random noise and then steadily generates an image. My study of the U-Net architecture has shown me how important the network's structure is: the down sampling layers capture general features, whereas the up-sampling layers reconstruct the spatial details. The skip connections pass information between the corresponding encoding and decoding layers and help the model retain image features during reconstruction.

I found that time conditioning indicates which diffusion step is currently being used, while class conditioning provides information about the type of image that is being requested. An important aspect was the need to examine the model carefully; even if the training loss is low, since this means that the noise prediction has been improved, it does not ensure that all the generated images are clear or identifiable. Therefore, a visual inspection is necessary to assess both image quality and class consistency. Furthermore, I found that CLIP offers an extra metric for evaluating how well an image matches a specific text description, although this score should be interpreted alongside other evidence.

This improved my coding and problem-solving skills because factors such as data normalization, tensor dimensions, device placement, and GPU memory all affect model performance. It is reasonable to test each component separately and to examine the intermediate results before training. When taking part in training. Understanding how my part fits into the whole notebook allow me to explain its purpose clearly.

To achieve better results in the future, I intend to implement pseudocode to explain the purpose of the main sections of the code and keep a record of the experimental conditions. I also hope to compare samples produced at different training stages and save checkpoints so I can track progress. By adopting these practices, I will be able to diagnose issues systematically and feel more confident when working on deep learning models.

Robert Thompson: Coding Improvements for the Diffusion Model Project

If I carried on with diffusion models, I would make them simpler to understand, simpler to verify and easier to reproduce. My emphasis would then be on organization, readability and making sure that the training is reproducible.

Divide the sampling into distinct components.

I need to break down the various stages—namely dataset preparation, the U-Net components, the noise schedule, training, and sampling—into separate functions or modules. The adjustable settings should be placed in a configuration file together with explanations of the tensor shapes and the reasons for the major operations. Doing so will make it easier to identify changes.

Make sure the tensors and the diffusion calculations are checked.

I need to check the image dimensions, the way the device is placed, the class labels, and the time-step ranges. Before carrying out full training, I must ensure that the shape of the predicted noise matches the shape of the target noise and that the loss is finite. I should also confirm that the forward noise formula and the reverse sampling updates use the correct schedule coefficients.

Improve the reliability of training and sampling.

Every time I carried out a training update, I cleared the gradients and kept the model, the optimizer, and the configuration details. When it came to sampling, I moved to evaluation mode and switched off the unnecessary tracking. If mixed precision was available, I would carefully consider the trade-offs between speed, memory usage, and numerical stability.

Make it possible to reproduce debugging and experiments.

Before carrying out several short experiments, I noted down the random seeds, the versions of the library, and the values, even though some of the GPU operations could still be nondeterministic. I included useful messages together with sample previews so that I would be able to examine any problems and modify a number of settings at the same time.

The aim in this instance is to give a more confident explanation and verification for each part of the code; these improvements would lead to a more reliable implementation and would make it easier for my teammates to review and extend the project.

Olis Bahari: Future Improvements for the Diffusion Model Project

In order to encourage research into diffusion models, I must focus on producing a reliable implementation, carrying out controlled training experiments, and carrying out a more accurate evaluation; the only way in which I will be able to find out why the model behaves as it does is by following this approach, and it will then be possible for me to make better decisions about image quality and the use of computational resources.

Look at the data and the model inputs.

Before carrying out the full training process, I would first run a small batch through the model in order to check the image normalization and the ordering of the class labels at each time step, making sure that the predictions matched the target noise shape. Taking this approach allows any implementation errors to be detected early on.

Compare training settings systematically.

I would alter just one parameter at a time, for instance the learning rate, the batch size, or the noise schedule, while keeping all the other settings unchanged. I maintained a record of the configurations, the training loss, and the examples; and as I saved checkpoints, I was able to restart the experiment from the beginning.

Examine the samples in each class.

For each class I set a predetermined number of samples; then I examined how easily they could be recognized, the variety of the samples, and the extent to which they matched the target class. I also visualized the intermediate stages of the denoising process and, using CLIP, looked at the similarity scores in conjunction with the visual evidence, since the scores by themselves do not account for all the aspects of image quality.

Manage the resources and enhance understanding.

I will begin by using a small dataset to create a simple model and then gradually increase the complexity as my resources permit. Should I find that the GPU memory is not sufficient, I will reduce the batch size before switching to a smaller model. At the same time, I also wish to examine the time embeddings and the reverse-sampling equations more carefully so as to be able to better explain and debug the process.